Pre-trained P-Code Representations for Cross-Architecture IoT Malware Detection via Graph Neural Networks

Date : 2026/02/24
Reporter: Yucheng Lin
Advisor: Shin-Ming Cheng, Han-Ming Lee

intelligent Systems Laboratory
台灣科大智慧型系統實驗室

Outline

1. Introduction
2. Related Work
3. Methodology
4. Experiments
5. Conclusion
6. References

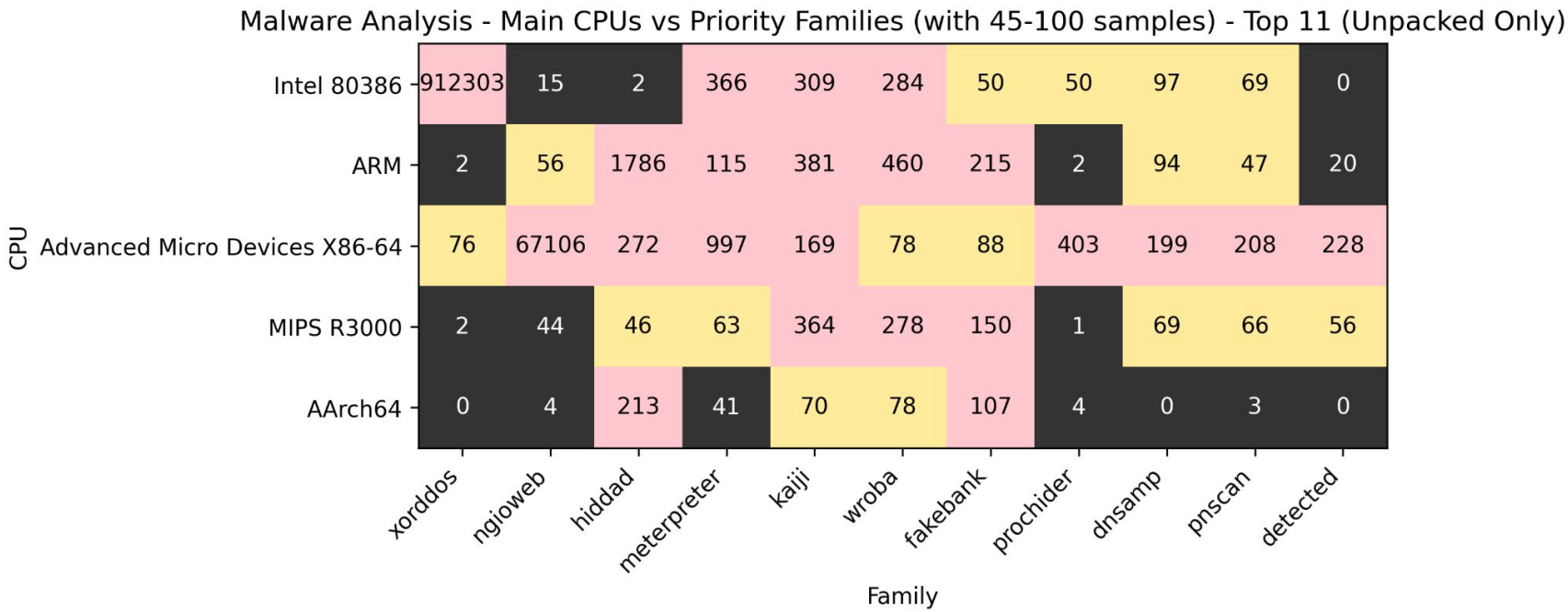
1. Introduction

Introduction: Background

- The IoT ecosystem is growing rapidly, with 21.1 billion devices in 2025 and over 40 billion expected by 2030 [1]
- This expansion brings security risks due to diverse architectures, weak authentication, and limited hardware resources [2, 3]
- IoT attacks rose from 32.7 million in 2018 to 112.29 million by 2022 [4]

Introduction: Background

- IoT devices across diverse CPU architectures creates security blind spots due to limited architecture-specific malware samples
- Leveraging abundant samples from common architectures to enhance detection across different CPU platforms.



Introduction: Popular Solution

- Feature: Printable String [9] , Intermediate Representation [8] [18], Attributed Control Flow Graph [10] [14]
- Method: Contrastive Learning [11] [12] [13], Transfer Learning
- Intermediate Representation: IDA Pro Microcode [16], Ghidra P-Code [6], Angr VexIR [17]

Introduction: Research Question

1. Leveraging abundant samples from common architectures to enhance detection across different CPU platforms.
2. Leverage Ghidra P-Code IR to bridge architectural differences, demonstrating its capability for robust cross-architecture analysis.
3. How can the integration of structural call graph features and semantic NLP embeddings improve the accuracy of cross-architecture malware detection?

Introduction: Ghidra P-Code

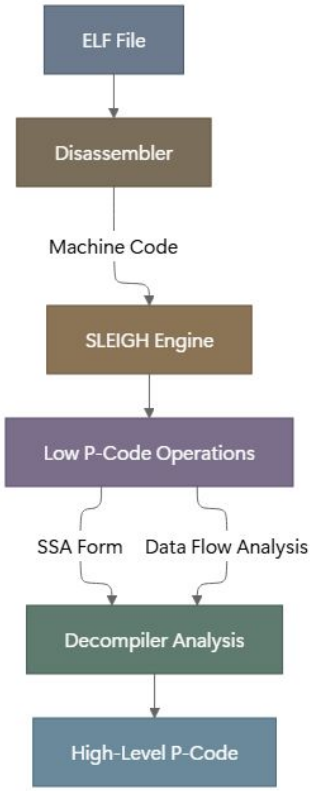
Ghidra: An open-source reverse engineering toolset developed by the NSA for disassembly and decompilation. [5]

P-Code Instruction: **"(ram, 0x5f10c, 4) COPY (ram, 0x5f10c, 4)"**

- **Address Space**: A generalization of RAM (Ex. RAM, Constant, Register)
- **Varnode**: A generalization of either a register or a memory location, include Address Space, Offset into the space and Size
- **P-code Operation**: The analog of a machine instruction [6]

Introduction: Ghidra P-Code

Although P-Code transforms diverse ISAs into a common high-level representation, it was never intended to be fully architecture-agnostic. [19]



C Source Code

```
int yyget_lineno  (void) { return yylineno ; }
```

Explanation: Get the current line number

Assembly Code

ARM-32	MIPS-32	x86-64

ldr r3,[0x11ea0]
ldr r0,[r3,#0x4]
bx lr
	lui v0,0x41
	jr ra
	_lw v0,0x5024(v0)
		MOV EAX,dword ptr [0x006050dc]
		RET

Low P-Code Operation

LOAD INT_ADD LOAD INT_AND INT_NOTEQUAL COPY INT_AND RETURN	INT_LEFT INT_AND INT_NOTEQUAL INT_2COMP INT_AND COPY INT_ADD COPY COPY LOAD RETURN INT_ADD COPY COPY LOAD	COPY INT_ZEXT LOAD INT_ADD RETURN

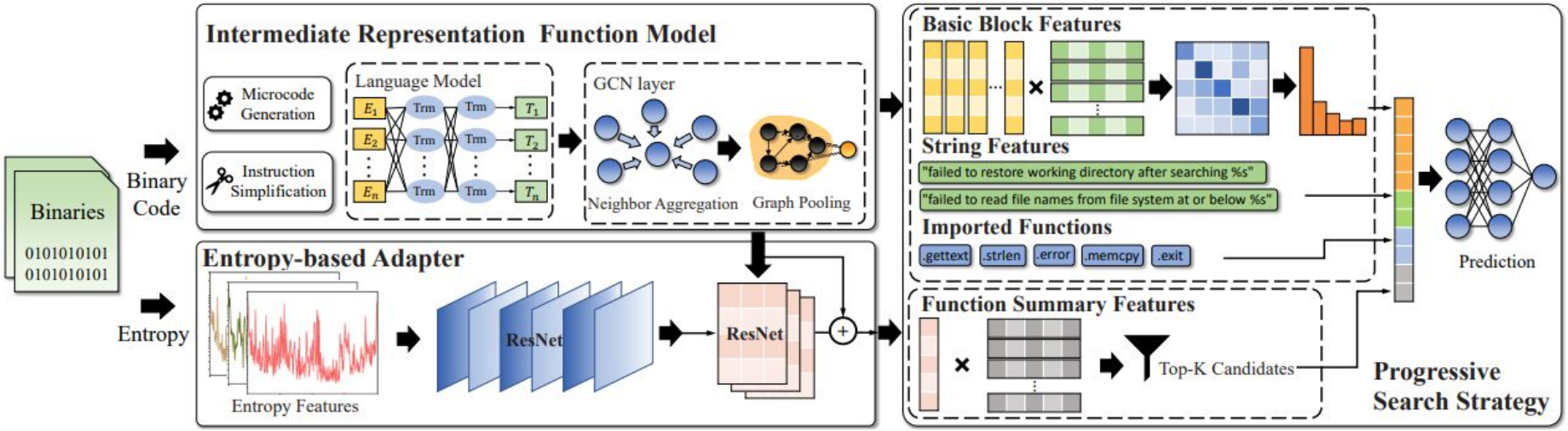
High P-Code Operation

INT_ADD LOAD CAST RETURN COPY	COPY RETURN COPY	RETURN COPY COPY

2. Related Work

Related Work

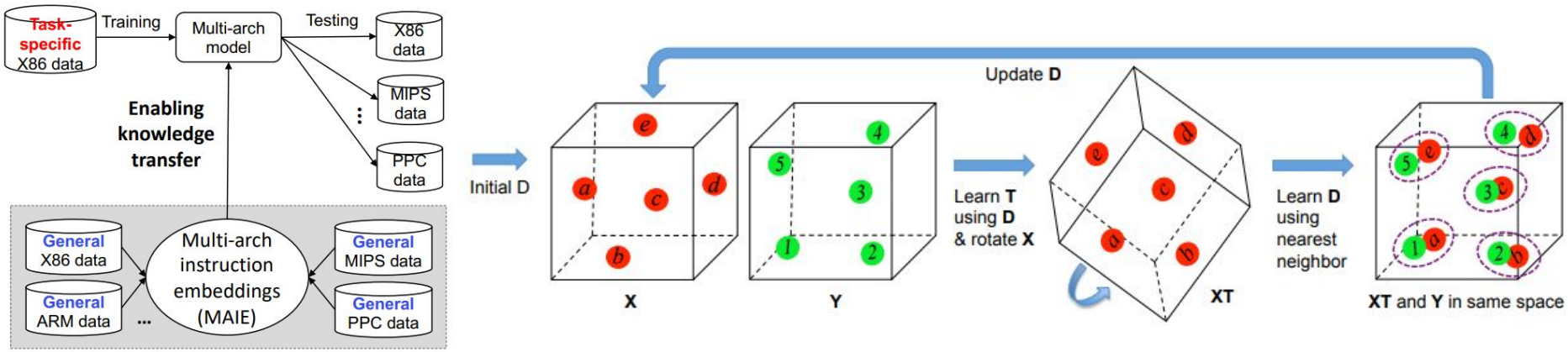
VulHawk uses entropy to identify a binary's compilation environment, and enhances vulnerability detection by calibrating function embeddings with strings and imported functions.



[7]

Related Work

Unimap achieves retargeted-architecture binary analysis by treating instructions as features and learning Multi-Architecture Instruction Embeddings (MAIE) to map similar instructions from different ISAs into a shared vector space.



[12]

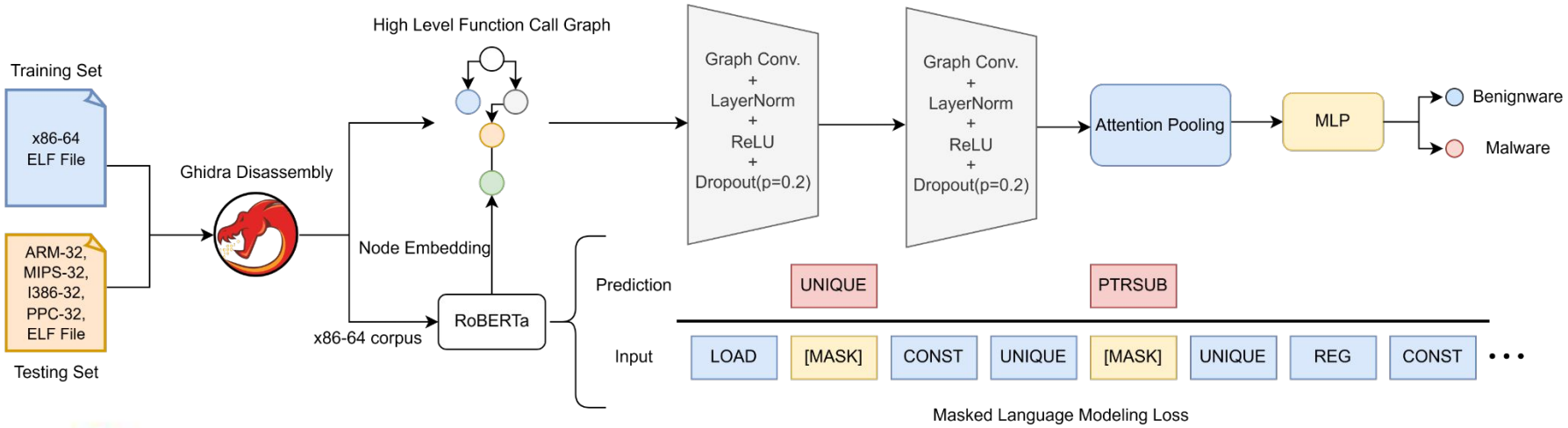
3. Methodology

Methodology

Embedding Model: RoBERTa

Graph Model:

- 2 layers GCN
- Loss: Cross Entropy
- Optimizer: Adam



Proposed Method Overview

Methodology

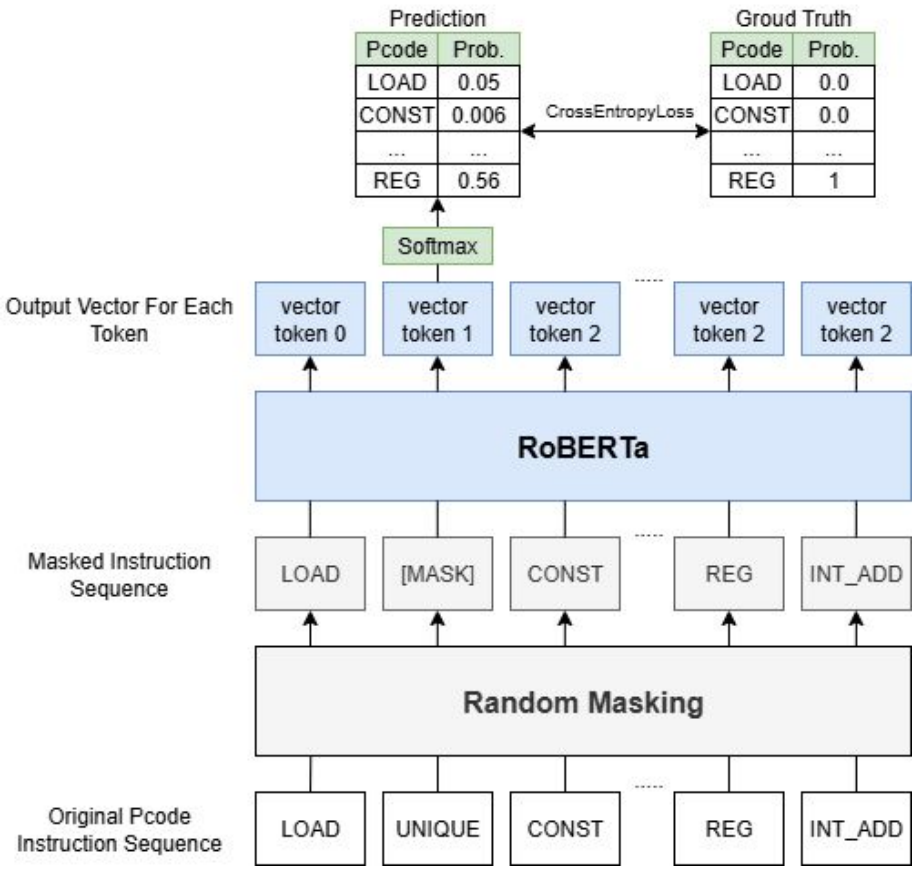
Function Preprocessing:

1. Remove thunk function and external function [15]
2. P-Code Normalization: (ram, 0x5f10c, 4) COPY (ram, 0x5f10c, 4) -> “RAM”, “COPY”, “RAM”
3. A function represents a node, and both P-Codes and Varnodes are treated as tokens

Methodology

Pre-training Parameters:

- vocab_size: 70(Pcode + Varnode)
- epochs: 100
- batch_size: 64
- lr: 0.0001
- random mask: 15%



4. Experiment

Experiments(Datasets)

- Cross Architecture Experiment Dataset

CPU vs Family Distribution

CPU	benign	dofloo	gafgyt	kaiji	meterpreter	mirai	mobidash	tsunami	wroba
ARM	1470	182	378	93	70	517	68	246	52
Intel	1478	346	390	46	67	515	50	242	16
MIPS	1618	111	439	176	38	550	141	267	45
PPC	1518	0	445	0	21	614	0	119	0
x86_64	1851	49	607	54	157	514	12	575	10

family

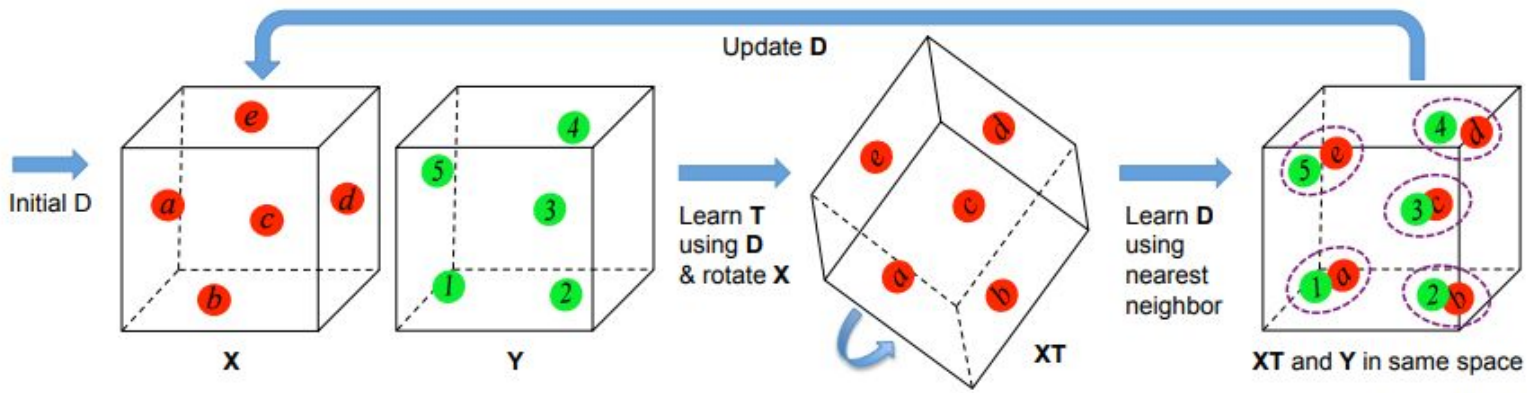
Experiments(Compare Different Embedding Method)

- Training data: x86_64
- Testing data: other CPU
- Performance Comparison: RoBERTa, CBOW, FastText, and Skip-gram

Embedding Method	Architecture	Accuracy (Mean ± SD)	AUC (Mean ± SD)	Precision (Mean ± SD)	Recall (Mean ± SD)	Test Samples
roberta	ARM	0.8532 ± 0.0426	0.9403 ± 0.0259	0.8619 ± 0.0393	0.8509 ± 0.0433	2987
roberta	PPC	0.9381 ± 0.0358	0.9692 ± 0.0366	0.9398 ± 0.0323	0.9353 ± 0.0393	2715
roberta	MIPS	0.6385 ± 0.0367	0.8265 ± 0.0198	0.7359 ± 0.0253	0.6562 ± 0.0339	3043
roberta	Intel	0.9539 ± 0.0165	0.9888 ± 0.0027	0.9551 ± 0.0153	0.9531 ± 0.0176	3125
cbow	ARM	0.6354 ± 0.0687	0.6678 ± 0.0713	0.6431 ± 0.0785	0.6330 ± 0.0694	2987
cbow	PPC	0.8019 ± 0.0814	0.8992 ± 0.0380	0.8306 ± 0.0637	0.7976 ± 0.0790	2715
cbow	MIPS	0.5748 ± 0.0404	0.6632 ± 0.0577	0.6681 ± 0.0555	0.5929 ± 0.0396	3043
cbow	Intel	0.6455 ± 0.0488	0.7655 ± 0.0905	0.6803 ± 0.0868	0.6497 ± 0.0555	3125
fasttext	ARM	0.7807 ± 0.0689	0.8489 ± 0.0458	0.7922 ± 0.0601	0.7792 ± 0.0709	2987
fasttext	PPC	0.7029 ± 0.1356	0.7274 ± 0.2200	0.7391 ± 0.1344	0.6707 ± 0.1551	2715
fasttext	MIPS	0.5834 ± 0.0405	0.6282 ± 0.0473	0.6420 ± 0.0396	0.5962 ± 0.0353	3043
fasttext	Intel	0.6071 ± 0.0837	0.8775 ± 0.1225	0.7142 ± 0.1432	0.6188 ± 0.0942	3125
skipgram	ARM	0.7195 ± 0.0986	0.7715 ± 0.0877	0.7245 ± 0.0999	0.7199 ± 0.0970	2987
skipgram	PPC	0.7078 ± 0.1349	0.7985 ± 0.0881	0.7402 ± 0.1070	0.7094 ± 0.1255	2715
skipgram	MIPS	0.5537 ± 0.0708	0.5541 ± 0.0629	0.6101 ± 0.1145	0.5645 ± 0.0803	3043
skipgram	Intel	0.5578 ± 0.1444	0.6318 ± 0.1563	0.5636 ± 0.1830	0.5600 ± 0.1534	3125

Experiments(Comparison with Other Works)

- Goal: $\mathbf{T} = \underset{\mathbf{T}}{\arg\min} \sum_i \sum_j \mathbf{D}_{ij} \|\mathbf{X}_{i*}\mathbf{T} - \mathbf{Y}_{j*}\|^2$
- Update T using gradient descent, and update D using cosine similarity



[12]

Experiments(Compare Different Embedding Method)

LSTM Parameter:

- Input Dimension: 200
- Hidden Size: 192
- Number of Layers: 2
- Dropout rate: 0.2
- Optimizer: Adam
- Epoch: 200
- Bidirectional: True

Dataset:

- x86-64 sample: 3,752
- ARM sample: 2,987

Source Arch	Target Arch	Accuracy (Mean ± SD)	AUC (Mean ± SD)	Precision (Mean ± SD)	Recall (Mean ± SD)
x86_64	ARM	0.7377 ± 0.0375	0.7848 ± 0.0352	0.7639 ± 0.0245	0.7141 ± 0.1178

Experiments(Single Architecture Performance)

- Model: RoBERTa, GCN
- Train/ Val/ Test: 7/1/2

Architecture	Accuracy (Mean ± SD)	Precision (Mean ± SD)	Recall (Mean ± SD)	F1 (Mean ± SD)
x86_64	0.9946 ± 0.0027	0.9947 ± 0.0026	0.9946 ± 0.0027	0.9946 ± 0.0027
ARM	0.9929 ± 0.0036	0.9936 ± 0.0032	0.9922 ± 0.0039	0.9929 ± 0.0036
MIPS	0.9841 ± 0.0018	0.9839 ± 0.0019	0.9843 ± 0.0015	0.9841 ± 0.0018
Intel	0.9968 ± 0.0000	0.9966 ± 0.0000	0.9970 ± 0.0000	0.9968 ± 0.0000
PPC	0.9963 ± 0.0037	0.9963 ± 0.0037	0.9963 ± 0.0037	0.9963 ± 0.0037

Current Progress: P-Code Normalization

Remove these P-Code:

- MULTIEQUAL: From the compiler theory concept of Static Single Assignment form, this is a phi-node
- INDIRECT: Operator copies input0 into output, but the value may be altered in an indirect way by the operation referred to by input1
- COPY: Copy a sequence of contiguous bytes from anywhere to anywhere
- CAST: Force a change in a value's data-type interpretation at this point in the code, without modifying the underlying value itself

Ref:https://github.com/NationalSecurityAgency/ghidra/blob/master/Ghidra/Features/Decompiler/src/decompile/cpp/ruleaction.cc

Current Progress: Graph features investigation

- Graph features distribution of our dataset:

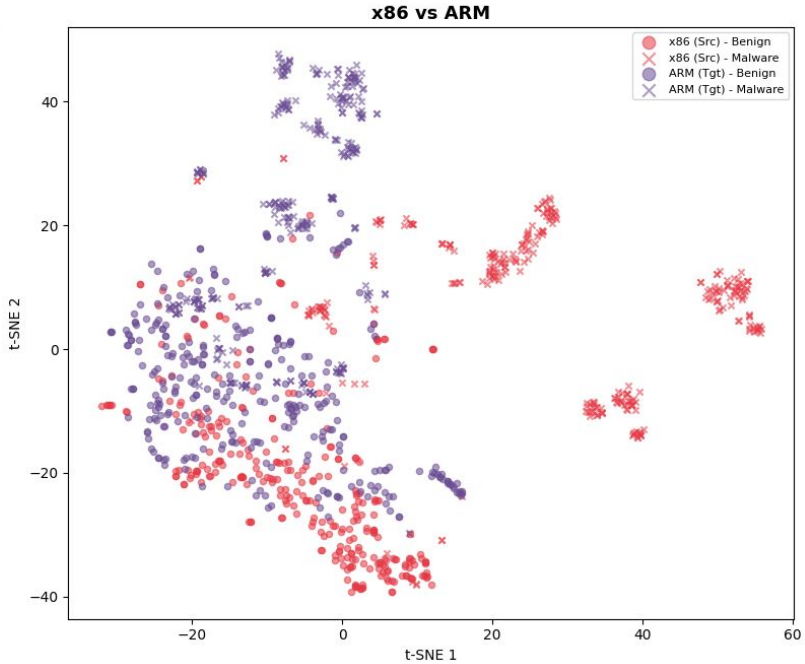
Architecture	Graphs	Edges (Mean)	Edges (Median)	Edges (Std)	Nodes (Mean)	Nodes (Median)	Nodes (Std)
x86_64	3751	609.3	75.0	1876.1	60.6	19.0	190.5
ARM	2987	877.4	432.0	1647.1	57.0	19.0	144.2
MIPS	3042	663.6	252.5	1535.8	46.1	16.0	120.1
Intel	3125	1540.3	567.0	2780.1	46.2	11.0	174.2
PPC	2715	854.8	408.0	1842.1	39.8	14.0	106.4

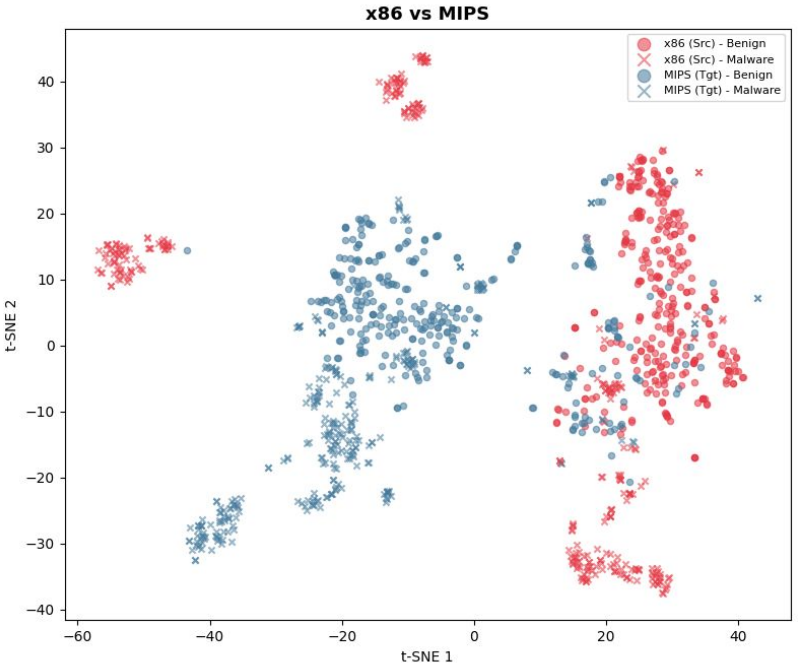
- Binkit Datasets [20]
- Density: Total Edges / Nodes*(Nodes-1)
- Degree: Total Edges / Total Nodes

Architecture	Binaries	Nodes (Mean)	Edges (Mean)	Avg Degree (Mean)	Density (Mean)
x86_64	100	326.77	592.97	1.23	0.0115
arm_32	100	237.15	345.36	1.19	0.0106
arm_64	100	262.35	449.04	1.18	0.0127
mips_32	99	408.88	575.02	1.26	0.0119

Current Progress: Word embedding results

- Each graph-level embedding is obtained by mean-pooling all node embeddings, then projected into 2D space using t-SNE for visualization





Current Progress: After Normalization

- Training data: x86_64
- Testing data: other CPU
- Performance Comparison: RoBERTa, CBOW and Skip-gram

Embedding	Target Arch	Accuracy	Precision	Recall	F1
RoBERTa	ARM	0.8269	0.8272	0.8279	0.8260
RoBERTa	MIPS	0.5713	0.7591	0.5889	0.5004
RoBERTa	Intel	0.8583	0.8665	0.8543	0.8576
RoBERTa	PPC	0.9086	0.9097	0.9113	0.9077
Skipgram	ARM	0.8822	0.8843	0.8807	0.8809
Skipgram	MIPS	0.5616	0.7527	0.5798	0.4822
Skipgram	Intel	0.9060	0.9033	0.9075	0.9077
Skipgram	PPC	0.8051	0.8440	0.7854	0.7810
CBOW	ARM	0.8405	0.8403	0.8436	0.8404
CBOW	MIPS	0.5705	0.7554	0.5884	0.4974
CBOW	Intel	0.8236	0.8412	0.8271	0.8222
CBOW	PPC	0.9061	0.9178	0.8976	0.9034

References

[1] S. Sinha. (2024) State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally. Accessed: 2026-02. [Online]. Available: https://iot-analytics.com/number-connected-iot-devices/

[2] A. D. Raju, I. Y. Abualhaol, R. S. Giagone, Y. Zhou, and S. Huang, "A survey on cross-architectural IoT malware threat hunting," IEEE Access, vol. 9, pp. 91686–91709, 2021.

[3] S. U. Qureshi, J. He, S. Tunio, N. Zhu, A. Nazir, A. Wajahat, F. Ullah, and A. Wadud, "Systematic review of deep learning solutions for malware detection and forensic analysis in IoT," Journal of King Saud University-Computer and Information Sciences, p. 102164, 2024.

References

[4] Ani Petrosyan, Statista (2024) Distribution of cyberattacks across worldwide industries in 2023. Accessed: 2026-02. [Online]. Available: https://www.statista.com/statistics/1315805/cyber-attacks-top industries-worldwide/

[5] Ghidra, "P-Code Reference Manual," 2023. Accessed: Feb. 2026. [Online]. Available: https://ghidra.re/ghidra_docs/languages/html/pcoderef.html

[6] National Security Agency, "Ghidra," version 11.0. Accessed: Feb. 2026. [Online]. Available: https://github.com/NationalSecurityAgency/ghidra

[7] LUO, Zhenhao, et al. VulHawk: Cross-architecture vulnerability detection with entropy-based binary code search. In: NDSS. 2023.

References

[8] PHU, Tran Nghi, et al. CFDVex: A novel feature extraction method for detecting cross-architecture IoT malware. In: Proceedings of the 10th international symposium on information and communication technology. 2019. p. 248-254.

[9] LEE, Yen-Ting, et al. Cross platform IoT-malware family classification based on printable strings. In: 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). IEEE, 2020. p. 775-784.

[10] FENG, Qian, et al. Scalable graph-based bug search for firmware images. In: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security. 2016. p. 480-491.

[11] REDMOND, Kimberly; LUO, Lannan; ZENG, Qiang. A cross-architecture instruction embedding model for natural language processing-inspired binary code analysis. arXiv preprint arXiv:1812.09652, 2018.

References

[12] WANG, Junzhe, et al. Can a deep learning model for one architecture be used for others?{Retargeted-Architecture} binary code analysis. In: 32nd USENIX Security Symposium (USENIX Security 23). 2023. p. 7339-7356.

[13] WANG, Junzhe; ZENG, Qiang; LUO, Lannan. Learning cross-architecture instruction embeddings for binary code analysis in low-resource architectures. In: Findings of the Association for Computational Linguistics: NAACL 2024. 2024. p. 1320-1332.

[14] YU, Shih-Yuan, et al. Cfg2vec: Hierarchical graph neural network for cross-architectural software reverse engineering. In: 2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, 2023. p. 281-291.

References

[15] LIN, Yu-Cheng, et al. Cross-Architecture IoT Malware Analysis Using P-Code Intermediate Representation. In: Proceedings of the 2025 Workshop on Recent Advances in Resilient and Trustworthy MAchine learning-driveN systems. 2025. p. 45-50.

[16] H. Rays, “IDA Pro,” https://www.hex-rays.com/products/ida/, May 2021

[17] Angr, "angr documentation," 2026. Accessed: Feb. 2026. [Online]. Available: https://docs.angr.io/en/latest/advanced-topics/ir.html

[18] RAHAT, Minhajul Alam, et al. Cimalir: Cross-platform iot malware clustering using intermediate representation. In: 2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC). IEEE, 2024. p. 0460-0466.

References

[19] NAUS, Nico, et al. A formal semantics for p-code. In: Working Conference on Verified Software: Theories, Tools, and Experiments. Cham: Springer International Publishing, 2022. p. 111-128.

[20] KIM, Dongkwan, et al. Revisiting binary code similarity analysis using interpretable feature engineering and lessons learned. IEEE Transactions on Software Engineering, 2022, 49.4: 1661-1682.